

passgen

Deterministic Password Generator

Детерминированный генератор паролей

Полная документация проекта для защиты

Описание клиентской части (index.html)

1. Идея проекта и решаемая проблема

В современном мире каждый человек сталкивается с проблемой паролей. У среднего пользователя 20-30 онлайн-аккаунтов. Для каждого требуется пароль.

Типичные решения имеют серьёзные недостатки:

1. Один пароль на все сервисы — опасно. При утечке базы данных одного сервиса злоумышленник получает доступ ко всем аккаунтам.
2. Запоминать все пароли — невозможно. Люди используют простые пароли (123456, password) или записывают их на стикерах.
3. Менеджеры паролей — создают единую точку отказа: если злоумышленник получит доступ к мастер-паролю, он получит все пароли сразу.

passgen предлагает четвёртый подход — детерминированную генерацию. Вместо хранения паролей программа каждый раз вычисляет их заново по криптографическому алгоритму. Пользователь запоминает всего одну фразу (сид), а пароль для любого сервиса восстанавливается по формуле: пароль = HMAC-SHA256(сид, сервис + длина + счётчик).

Преимущества:

- Не нужно хранить пароли — они всегда восстанавливаются из сида и названия сервиса
- Нечего красть — на компьютере нет базы данных с паролями
- Нельзя потерять — сид можно восстановить, если записать его в надёжном месте
- Разные пароли для каждого сервиса — достаточно изменить название
- Не зависит от интернета — вся генерация происходит локально в браузере
- Бесплатно и без рекламы — программа с открытым исходным кодом

2. Структура клиентской части (index.html)

Вся программа находится в одном файле — index.html. Он содержит три компонента: HTML-разметку, CSS-стили и JavaScript-логику. Никаких файлов Python, серверной части или баз данных не требуется. Файл открывается в любом современном браузере.

2.1. HTML — разметка страницы

HTML определяет структуру интерфейса. Все элементы сгруппированы в блоки (карточки):

1. Верхняя панель: заголовок "passgen" слева, ссылки "Экспорт" и "Импорт" справа.
2. Блок "Сид": поле ввода секретной фразы, место для сообщения об ошибке.
3. Блок "Сервис": поле ввода названия сервиса, кнопка "список" с панелью чипов.
4. Блок "Параметры": пресеты (выпадающий список), длина пароля со стрелками, 4 чекбокса наборов символов, расширенные настройки (инверсия).
5. Кнопка "Создать".
6. Блок "Логин и пароль": поле логина с кнопкой "Создать логин", поле пароля, индикатор надёжности.
7. Кнопка "Копировать".
8. Модальное окно импорта (скрыто по умолчанию).

2.2. CSS — стили и дизайн

CSS определяет внешний вид программы. Ключевые особенности:

Цветовая схема задаётся через CSS-переменные (:root):

```
--bg: #f5f3ff (светло-фиолетовый фон страницы)
--card: #ffffff (белый фон карточек)
--fg: #1a1a2e (тёмно-синий текст)
--border: #e5e7eb (светло-серая граница)
--lilac: #a855f7 (фиолетовый — основной акцентный цвет)
--lilac2: #7c3aed (тёмно-фиолетовый для градиентов)
--pink: #f472b6 (розовый — слабый пароль)
--crimson: #e11d48 (малиновый — средний пароль)
--error: #f43f5e (красный — ошибка)
```

Анимации:

```
@keyframes fadeUp — плавное появление элементов (сдвиг 8px вверх, прозрачность 0->1)
@keyframes glow — пульсация границы при ошибке (розовый <-> красный)
@keyframes slideDown/slideUp — раскрытие/сворачивание панелей (max-height 0->300px)
@keyframes checkPop — эффект пружинки при переключении чекбокса (scale 1->1.2->1)
```

Кнопки: градиентный фон (linear-gradient), скруглённые углы (border-radius: 11px), тень, анимация нажатия (scale: 0.97).

Карточки: белый фон, скруглённые углы (12px), тень (box-shadow), отступы (padding: 14px 16px).

Скроллбар скрыт (::-webkit-scrollbar { display: none }), прокрутка работает.

2.3. JavaScript — логика программы

JavaScript реализует всю логику: генерацию паролей, сохранение настроек, экспорт/импорт, обработку событий. Используется Web Crypto API — встроенная в браузер библиотека криптографических функций. Никаких внешних библиотек или пакетов не требуется.

3. Криптографический алгоритм (детально)

В этом разделе подробно объясняется, как программа вычисляет пароль. Это ключевая часть для понимания работы программы.

3.1. Общая схема

Входные данные:

- seed (сид): секретная фраза, введённая пользователем
- service (сервис): название сайта или приложения
- length (длина): желаемое количество символов в пароле
- charset (алфавит): строка со всеми допустимыми символами

Выходные данные:

- password (пароль): строка длины length, состоящая из символов charset

Дополнительные опции:

- capitalize (первая заглавная): сделать первый символ заглавным
- lastDigit (последняя цифра): заменить последний символ на цифру

3.2. Подробная схема HMAC-генерации

1. Сид преобразуется в массив байтов через `TextEncoder`.
2. Из этих байтов создаётся криптографический ключ через `crypto.subtle.importKey` с алгоритмом `{name: "HMAC", hash: "SHA-256"}`.
3. Формируется базовая часть сообщения: сервис + длина (как строка).
4. Запускается цикл сбора символов:
 - a. К базовому сообщению добавляется счётчик (`ct = 0, 1, 2...`)
 - b. Вычисляется HMAC-SHA256: `crypto.subtle.sign("HMAC", key, message)`
 - c. Из полученных 32 байтов каждый проверяется:
 - Если байт $< (256 - (256 \% \text{len}(\text{charset})))$, он подходит
 - Подходящий байт: `символ = charset[байт \% \text{len}(\text{charset})]`
 - d. Счётчик увеличивается, пока не наберётся length символов
5. Готовый пароль возвращается.

3.3. Rejection Sampling

Критически важная деталь. 256 (количество значений байта) не делится нацело на длину алфавита. Если бы программа просто брала `байт \% \text{len}(\text{charset})` для всех байтов, первые символы алфавита встречались бы чаще последних. Это смещение (bias) снижает криптостойкость.

Решение: программа вычисляет $\text{max} = 256 - (256 \% \text{len}(\text{charset}))$. Байты $\geq \text{max}$ отбрасываются. Берутся только байты $< \text{max}$ — для них распределение будет равномерным. Пример: алфавит из 50 символов: $\text{max} = 256 - 6 = 250$. Байты 250-255 отбрасываются. Каждый символ встречается ровно 5 раз (250/50).

3.4. Инверсия: первая заглавная и последняя цифра

Первая заглавная: первый символ пароля преобразуется в верхний регистр через метод `toUpperCase()`. Если символ уже заглавный — ничего не меняется.

Последняя цифра: вычисляется отдельный HMAC(сид+"_lastdig", сервис+длина). Из первого байта результата

берётся цифра = байт % 10. Последний символ пароля заменяется на эту цифру. Отдельный HMAC нужен, чтобы не нарушить детерминированность основного пароля.

3.5. Алфавиты наборов символов

Заглавные: ABCDEFGHJKLMNPQRSTUVWXYZ

(исключены I, O — путаются с 1 и 0)

Строчные: abcdefghikmnpqrstuvwxy

(исключены j, l, o — путаются с цифрами)

Цифры: 0123456789

Спецсимволы: !@#\$%&*+=-.

По умолчанию включены: заглавные, строчные, спецсимволы. Длина по умолчанию: 10 символов.

3.6. Генерация логина

Логин генерируется через SHA-256 (не HMAC). Вход: сид + "login" + сервис + счётчик (loginCounter, увеличивается при каждом вызове). Из 32 байтов хеша берутся первые 6. Алфавит логина: "abcdefghijklmnopqrstuvwxyz23456789" (нет i,l,o — путаются с 1,0; нет 0,1 — путаются с буквами). Каждый клик даёт новый логин благодаря счётчику.

4. Интерфейс программы (полный обзор)

Описание каждого элемента пользовательского интерфейса.

4.1. Верхняя панель

Слева — заголовок "passgen". Справа — текстовые ссылки "Экспорт" (скачать JSON) и "Импорт" (загрузить JSON). Импорт использует скрытый input type="file".

4.2. Блок сида

Заголовок "Сид". Подпись: "Секретная фраза, которую вы держите в голове." Поле ввода с placeholder "Ваша секретная фраза". Под полем — место для ошибки (розовый текст). При пустом сиде — пульсирующая розовая граница (анимация glow).

4.3. Блок сервиса

Заголовок "Сервис", справа кнопка "список" со стрелкой. Поле ввода с placeholder "google, github, vk ...". Под полем — скрытая панель с чипами сервисов, открывается по клику.

4.4. Список сервисов (чипы)

Популярные сервисы с цветными кружками-логотипами:

Google (синий G), GitHub (чёрный GH), VK (синий V), Яндекс (красный Ya), Mail (синий @), iCloud (тёмно-синий i), Dropbox (синий D), Telegram (голубой T).

Пользовательские сервисы — с серым кружком и "?". Клик по чипу — заполняет поле сервиса и применяет настройки. Правый клик на сохранённом — удаляет его.

4.5. Блок параметров

Выпадающий список пресетов:

- "Свой" (ручная настройка)
- "Минимальный (8, буквы+цифры)" — длина 8, capitals+lowercase+digits
- "Стандартный (10, всё кроме спецсимволов)" — длина 10, capitals+lowercase+digits
- "Максимальный (16, всё)" — длина 16, все 4 набора

Длина пароля (input type="number" 1-99) со стрелками вверх/вниз.

4 чекбокса-кружочка для наборов символов.

Ссылка "Расширенные" — раскрывает опции "Первая заглавная" и "Последняя цифра".

4.6. Блок результата

Поле логина (только чтение) с кнопкой "Создать логин".

Поле пароля (только чтение) — моноширинный шрифт, жирный, по центру.

Индикатор надёжности: 3 полоски + текстовая рекомендация.

4.7. Кнопки

"Создать" — градиент lilac->lilac2, белый текст, тень. Запускает генерацию.

"Копировать" — градиент violet->#8b5cf6. Копирует пароль.

Обе кнопки с анимацией нажатия (scale 0.97) и появления (fadeUp).

4.8. Модальное окно импорта

Появляется поверх программы при импорте JSON. Оверлей с центрированным блоком. Заголовок, список сервисов с чекбоксами, кнопки "Отмена" и "Импортировать".

5. Все функции JavaScript (подробно)

Описание каждой функции программы. Текст написан так, чтобы человек без знания программирования мог понять, что делает программа и как она работает.

5.1. init() — подготовка программы к работе

Когда пользователь открывает страницу, нужно подготовить все поля и кнопки. `init()` выполняется автоматически при загрузке (последняя строка в JavaScript: `init()`).

`init()` сначала очищает все старые данные (вызывает `loadState()`). Убирает возможные ошибки с поля сида. Привязывает обработчики: при вводе сида ошибка исчезает; при Enter в поле сида или сервиса — запускается генерация; при потере фокуса на поле сервиса — программа запоминает, какой сервис выбран.

Затем `init()` проверяет, есть ли настройки для текущего сервиса. Если есть — восстанавливает их. Если нет — устанавливает стандартные: заглавные+строчные+специальные символы, длина 10. После этого отрисовывает список сервисов с логотипами.

5.2. loadState() — очистка данных (безопасность)

Эта функция гарантирует, что при каждом запуске программа начинается с чистого листа. Удаляет всё из памяти браузера, обнуляет список сервисов и настройки, сбрасывает длину на 10, очищает поле сервиса. Ключевой элемент безопасности — никто не увидит, для каких сервисов вы генерировали пароли.

5.3. saveCfg() — сохранение настроек сервиса

Когда пользователь нажимает "Создать", программа запоминает текущие настройки для этого сервиса. saveCfg() берёт состояние чекбоксов (какие наборы включены), опции инверсии и длину, и сохраняет в perSvc под названием сервиса.

Настройки хранятся только в оперативной памяти. При закрытии страницы исчезают.

5.4. applyCfg(cfg) — восстановление настроек

Когда пользователь выбирает сервис из списка, applyCfg() восстанавливает его настройки: устанавливает длину, включает/выключает чекбоксы и опции инверсии, перерисовывает интерфейс.

5.5. toggleInv(id) — переключение расширенных опций

Клик по "Первая заглавная" или "Последняя цифра" переключает их состояние (0->1, 1->0). Функция обновляет кружочек на экране. Настройки не сохраняются до нажатия "Создать".

5.6. applyPreset() — применение пресета

При выборе пресета из списка: Минимальный (длина 8, capitals+lowercase+digits), Стандартный (длина 10, capitals+lowercase+digits), Максимальный (длина 16, все 4 набора). Если выбран "Свой" — функция ничего не делает.

5.7. renderChk() — отрисовка чекбоксов

Создаёт 4 строки с кружочками и названиями наборов. Если набор включён — кружок фиолетовый. При клике значение переключается, функция запускается заново для обновления экрана.

5.8. renderInv() — отрисовка инверсии

Обновляет кружочки для первой заглавной и последней цифры. Просто добавляет или убирает класс "on".

5.9. adj(delta) — изменение длины

При клике на стрелки (+1/-1) меняет длину пароля. Проверяет границы: от 1 до 99.

5.10. toggleSvcPanel() — открытие списка сервисов

Переключает CSS-класс "open" на панели сервисов. Когда класс есть — панель видна (max-height: 300px), когда нет — скрыта (max-height: 0). Стрелка поворачивается.

5.11. renderSvcChips() — сборка списка сервисов

Сначала показывает популярные сервисы из предустановленного списка (которых нет в сохранённых), затем все сохранённые сервисы пользователя. Для каждого вызывает `chip()`.

5.12. chip() — создание чипа сервиса

Создаёт кнопку с логотипом. Для популярных сервисов — цветной кружок с буквой (Google = синий G). Для пользовательских — серый кружок с "?". При клике — заполняет поле сервиса и применяет настройки. Правый клик — удаляет сервис.

5.13. gen() — генерация пароля (главная функция)

Самая важная функция. Запускается при нажатии "Создать" или Enter.

Шаг за шагом:

1. Проверяет сид — если пустой, розовая ошибка и останов.
2. Проверяет чекбоксы — если ни один не включён, ошибка.
3. Составляет алфавит из выбранных наборов.
4. Вызывает hmacGen() для вычисления HMAC-SHA256.
5. Применяет первую заглавную (если включена).
6. Применяет последнюю цифру (если включена).
7. Показывает пароль на экране.
8. Обновляет индикатор надёжности.
9. Копирует пароль в буфер обмена.
10. Добавляет сервис в список (если его не было).
11. Сохраняет настройки.

5.14. hmacGen() — криптографическое ядро

Выполняет actual вычисление HMAC-SHA256. Асинхронная функция (async), чтобы браузер не зависал.

Алгоритм:

1. Сид -> байты -> ключ HMAC (crypto.subtle.importKey).
2. Сообщение: сервис + длина (как байты).
3. $\text{max} = 256 - (256 \% \text{длина_алфавита})$ для rejection sampling.
4. Цикл: пока не наберётся нужная длина:
 - Добавляет счётчик к сообщению
 - Вычисляет HMAC (32 байта)
 - Для каждого байта: если $< \text{max}$, добавляет символ в пароль
 - Счётчик ++
5. Возвращает пароль.

5.15. genLogin() — генерация логина

Создаёт логин из 6 символов. Проверяет сид и сервис. Увеличивает счётчик loginCounter. Вычисляет SHA-256 от сид+"login"+сервис+счётчик. Берёт первые 6 байтов, превращает в символы из алфавита "abcdefghijklmnopqrstuvwxyz23456789". Каждый клик — новый логин (благодаря счётчику).

5.16. updateStrength() — индикатор надёжности

Анализирует длину и количество наборов:

- < 10 символов или < 3 наборов: слабый (розовый, 1 полоска)
- >= 10 и >= 3: средний (малиновый, 2 полоски)
- >= 14 и = 4: отличный (сиреневый, 3 полоски)

Показывает текст с рекомендацией и эмодзи (>.< / :3 / uwu).

5.17. `cpy()` — копирование в буфер

Копирует пароль в буфер обмена через `navigator.clipboard.writeText()`. При ошибке использует `execCommand("copy")`. Показывает "Скопировано!" на 1.5с.

5.18. exportData() — экспорт в JSON

Собирает все сервисы с настройками, форматирует в JSON (с отступами 2 пробела), создаёт Blob, программно кликает по ссылке для скачивания файла `passgen_export.json`.

5.19. importData(event) — импорт из JSON

Читает выбранный пользователем JSON-файл через FileReader. Парсит и проверяет. Если данные корректны — открывает модальное окно выбора сервисов.

5.20. showImportModal(d) — окно выбора сервисов

Создаёт список всех сервисов из файла с чекбоксами (все включены). Если файл пуст — сообщение об ошибке.

5.21. confirmImport() — подтверждение импорта

Берёт отмеченные сервисы, загружает настройки в perSvc, добавляет в svcs, сортирует, перерисовывает чипы. Показывает "Импорт завершён!" на 3 секунды.

5.22. closeImportModal() — закрытие окна импорта

Убирает модальное окно, очищает временные данные.

5.23. toggleAdvanced() — расширенная панель

Раскрывает/сворачивает панель с опциями инверсии через CSS (max-height 0->120px).

5.24. getCfg(svc) — получение настроек

Если для сервиса нет настроек — создаёт стандартные. Возвращает настройки.

5.25. onSvcChange() — смена сервиса

Обновляет curSvc и применяет настройки, если есть.

5.26. saveState() — заглушка

Пустая функция. Оставлена для возможного будущего использования. В текущей версии ничего не сохраняет (сессионный режим).

6. Обработка ошибок

1. Пустой сид — розовая подсветка, сообщение "Введите сид", генерация не происходит. При вводе ошибка снимается.
2. Нет набора символов — сообщение "Выберите хотя бы один тип символов".
3. Длина вне диапазона (1-99) — `adj()` принудительно ставит границу.
4. Ошибка импорта (неверный файл) — сообщение об ошибке.
5. Пустой импорт (нет сервисов) — "Нет сервисов для импорта".
6. Ошибка копирования — запасной метод `exesCommand("copy")`.

7. Экспорт и импорт (формат JSON)

Пример структуры файла:

```
{
  "svcs": ["google", "github", "vk"],
  "perSvc": {
    "google": {
      "state": {"capitals":1,"lowercase":1,"digits":0,"symbols":1},
      "inv": {"cap":0,"dig":1},
      "len": 16
    }
  }
}
```

svcs — массив названий сервисов.

perSvc — объект с настройками для каждого:

state — флаги capitals, lowercase, digits, symbols (1 = включено)

inv — флаги cap (первая заглавная), dig (последняя цифра)

len — длина пароля (1-99)

8. Безопасность

1. Локальность: всё работает в браузере на localhost, данные не уходят в сеть.
2. Сессионный режим: настройки только в оперативной памяти, при перезагрузке всё очищается.
3. Стандартная криптография: HMAC-SHA256 — международный стандарт (RFC 2104), используется в банках, блокчейне, TLS/SSL.
4. Rejection sampling: равномерное распределение символов, без смещения вероятности.
5. Разделение ключа и сообщения: зная пароль, нельзя восстановить сид (необратимость хеш-функции).
6. Разные пароли для разных сервисов: название сервиса — часть входных данных HMAC.

Чего программа НЕ делает

- Не хранит пароли (вычисляет их заново)
- Не использует соль (роль соли выполняет название сервиса)
- Не защищает от кейлоггеров
- Не требует мастер-пароля (сид — это мастер-пароль)
- Не создаёт резервных копий (сид храните отдельно)

9. Запуск программы

Программа — это один файл `index.html`. Его можно открыть любым способом:

Способ 1: дважды кликнуть по файлу `index.html` — откроется в браузере.

Способ 2: открыть браузер, нажать `Ctrl+O`, выбрать `index.html`.

Способ 3: если есть Python, можно запустить `app.py` — он откроет окно в режиме приложения (без адресной строки). Но это не обязательно — `index.html` работает сам по себе.

Системные требования:

- Любой современный браузер (Chrome, Firefox, Edge, Safari)
- Web Crypto API (есть во всех современных браузерах)
- Интернет не требуется

Важно: файл должен открываться через протокол `http://` или `https://`, а не `file://`, чтобы работал Web Crypto API. При открытии через `file://` криптография может быть недоступна. Поэтому рекомендуется использовать простой HTTP-сервер (например, `app.py` или `python3 -m http.server`).